

try / catch / finally

```
try {
    // código que puede fallar
} catch (TipoExcepcion e) {
    // manejo del error
} finally {
    // se ejecuta SIEMPRE
    // (se lance o no excepción)
}
```

- **try:** bloque a vigilar
- **catch:** captura y maneja el error
- **finally:** opcional, para cerrar recursos, logs...

throw / throws

```
// LANZAR una excepción
throw new IllegalArgumentException(
    "edad no válida");

// DECLARAR que un método
// puede lanzarla
public void leer(String f)
    throws IOException {
    ...
}
```

- **throw:** lanza el objeto excepción (dentro del método)
- **throws:** avisa en la firma de que puede saltar (obligatorio si es checked)

Multi-catch

```
try {
    ...
} catch (IOException
    | SQLException e) {
    e.printStackTrace();
}
```

Úsalo cuando el mismo bloque de código debe reaccionar igual ante varios tipos de excepción.

try-with-resources

```
try (FileReader fr =
    new FileReader("a.txt")) {
    // usar fr
} catch (IOException e) {
    e.printStackTrace();
}
// fr se cierra solo (AutoCloseable)
```

Úsalo con ficheros, sockets, BD... evita olvidarte del `close()`.

Excepción personalizada

```
public class SaldoException
    extends Exception {
    public SaldoException(String
        msg) {
        super(msg);
    }
}
// uso:
if (saldo < 0)
    throw new SaldoException(
        "saldo insuficiente");
```

Extiende `Exception` (checked) o `RuntimeException` (unchecked) según si quieres obligar a capturarla.

Checked (obligan a try/catch o throws)

<code>IOException</code>	Error de entrada/salida (leer fichero, red...)
<code>FileNotFoundException</code>	El fichero indicado no existe
<code>SQLException</code>	Error al acceder/consultar una base de datos
<code>ClassNotFoundException</code>	No encuentra la clase al cargarla dinámicamente
<code>InterruptedException</code>	Un hilo (Thread) es interrumpido mientras espera
<code>ParseException</code>	Error al parsear texto (fechas, formatos...)

Jerarquía (de arriba abajo)

- `Throwable`
- `↳ Error` (fallos graves de la JVM, NO se capturan normalmente)
- `↳ Exception`
- `↳ Checked` (todas menos `RuntimeException`)
- `↳ RuntimeException` → Unchecked

Checked: el compilador te OBLIGA a capturarla o declararla (throws).
Unchecked: no obliga; suelen ser errores de programación.

Buenas prácticas

- Captura la excepción más específica primero, la genérica al final.
- Nunca dejes un `catch` vacío: como mínimo, registra el error (log).
- No uses excepciones para el flujo normal del programa.
- Cierra recursos siempre → usa `try-with-resources`.
- Crea excepciones propias solo si aportan info de negocio real.
- `e.getMessage()` → mensaje;
`e.printStackTrace()` → traza completa.

Unchecked / RuntimeException (no obligan)

<code>NullPointerException</code>	Usas un objeto que vale <code>null</code>
<code>ArithmeticException</code>	Operación matemática inválida (ej. / 0 con enteros)
<code>ArrayIndexOutOfBoundsException</code>	Índice de array fuera de rango
<code>StringIndexOutOfBoundsException</code>	Índice de String fuera de rango
<code>NumberFormatException</code>	Convertir texto no numérico a número (parseInt...)
<code>ClassCastException</code>	Casting inválido entre tipos incompatibles
<code>IllegalArgumentException</code>	Argumento pasado a un método no es válido
<code>IllegalStateException</code>	Se llama a un método en un momento/estado incorrecto
<code>UnsupportedOperationException</code>	Operación no soportada (ej. lista inmutable)
<code>ConcurrentModificationException</code>	Modificar una colección mientras se recorre
<code>NegativeArraySizeException</code>	Crear un array con tamaño negativo

Orden correcto de catch

```
try {
    ...
} catch (FileNotFoundException e) {
    // más específica primero
} catch (IOException e) {
    // más genérica después
} catch (Exception e) {
    // comodín, al final
}
```

Si pones la genérica primero, el compilador da error: código inalcanzable.